Báo cáo Bài 3
Trao đổi dữ liệu bằng AMQP

Phần 1 Bài lab trên lớp theo nhóm
Mã nhóm 16

Thành viên nhóm

STT	Họ và tên	MSSV	Vai trò
1	Nguyễn Đăng Long	202400057	Thành viên
2	Nguyễn Hoàng Gia	202400040	Nhóm trưởng
3	Nguyễn Trí Hiếu	202416203	Thành viên
4	Phạm Gia Linh	202416262	Thành viên

Lớp ..... Ngày thực hiện 23 tháng 10 năm 2026

1 Mục tiêu và mô hình hệ thống

Nhóm 16 xây dựng hệ thống mô phỏng cảm biến gồm một producer, một RabbitMQ broker và một consumer. Producer gửi nhiệt độ và độ ẩm dưới dạng JSON mỗi 5 giây; consumer nhận, phân tích và hiển thị dữ liệu. Toàn bộ hệ thống chạy bằng Docker Compose trên một máy.

Kết quả kiểm tra cho thấy queue giữ bản tin trong thời gian consumer dừng và chuyển tiếp bản tin đang chờ khi consumer chạy lại. Báo cáo có bốn ảnh chụp thực nghiệm: chương trình chạy, queue bình thường, queue tích bản tin và queue sau khi nhận hết.

```
Producer --> default exchange --> it4735.nhom16.telemetry --> Consumer
                routing key = queue name                <-- ACK
```

Thông số	Giá trị của nhóm 16
Giao thức	AMQP 0-9-1 qua thư viện Pika
Queue và routing key	it4735.nhom16.telemetry
Exchange	Default exchange có tên là chuỗi rỗng
Bản tin mẫu	{"temperature": 23, "humidity": 33}
Chu kỳ và đơn vị	5 giây; nhiệt độ °C; độ ẩm %
Broker và giao diện web	rabbitmq:5672 nội bộ; localhost:15672 trên máy

Cảm biến trong bài là dữ liệu mô phỏng, không sử dụng phần cứng thật. Báo cáo Word được nộp riêng; ZIP đi kèm chỉ chứa mã nguồn và cấu hình Docker.

Cấu trúc gói mã nguồn

```
bai3_1/
  common.py                # cau hinh RabbitMQ va queue
  producer.py, consumer.py  # gui va nhan thong diep
  compose.yaml, Dockerfile  # RabbitMQ va cac container
  requirements.txt          # thu vien pika
  .env.example, .dockerignore # cau hinh mau va loai file
```

2 Producer và cách gửi bản tin

common.py đọc hostname và tài khoản từ biến môi trường, tạo BlockingConnection và khai báo queue. Cả producer và consumer đều gọi cùng hàm này để tránh khác thuộc tính queue.

```
QUEUE_NAME = os.getenv("QUEUE_NAME", "it4735.nhom16.telemetry")

def declare_queue(channel):
    return channel.queue_declare(queue=QUEUE_NAME, durable=True)
```

Trong producer.py, các dòng chính trong vòng lặp gửi như sau. Biến interval mặc định bằng 5, lấy từ INTERVAL_SECONDS. Phần thiết lập kết nối và xử lý dừng nằm trong mã nguồn đính kèm.

```
payload = {"temperature": round(random.uniform(23, 35), 1),
           "humidity": round(random.uniform(40, 80), 1)}
body = json.dumps(payload, allow_nan=False).encode("utf-8")
channel.basic_publish(
    exchange="",  # Default exchange: routing key chinh la ten queue.
    routing_key=QUEUE_NAME,
    body=body,
    properties=pika.BasicProperties(
        content_type="application/json", content_encoding="utf-8",
        delivery_mode=2),
    mandatory=True,
)
```

Các giá trị nhiệt độ và độ ẩm được sinh ngẫu nhiên, làm tròn một chữ số thập phân rồi chuyển thành JSON và mã hóa UTF-8. Trong cấu hình này, nhiệt độ nằm trong khoảng 23 đến 35 °C và độ ẩm từ 40 đến 80%.

exchange="" chọn default exchange có sẵn của RabbitMQ. RabbitMQ tự tạo binding giữa default exchange và mỗi queue, với routing key bằng tên queue. Vì vậy producer gửi trực tiếp tới it4735.nhom16.telemetry mà không cần tự khai báo exchange hoặc binding.

durable=True giữ khai báo queue khi broker khởi động lại; delivery_mode=2 đánh dấu bản tin persistent. Docker volume giữ dữ liệu broker khi tạo lại container. channel.confirm_delivery() bật xác nhận từ broker; mandatory=True phát hiện trường hợp bản tin không được định tuyến tới queue.

Sau mỗi lần gửi, chương trình ghi log SENT rồi chờ đủ chu kỳ bằng connection.sleep theo từng khoảng ngắn. Cách chờ này vẫn xử lý heartbeat và phản hồi tín hiệu dừng. Khi lỗi kết nối, ứng dụng chờ 5 giây rồi kết nối lại. Cấu hình này không bảo đảm xử lý đúng một lần trong mọi sự cố mạng.

3 Consumer và cách xác nhận bản tin

Consumer khai báo cùng queue, đặt prefetch_count=1 và sử dụng xác nhận thủ công. Callback on_message xử lý từng bản tin như sau.

```
def on_message(channel, method, properties, body):
    try:
        temperature, humidity = parse_telemetry(body)
    except (ValueError, UnicodeDecodeError, OverflowError) as exc:
        log.error("JSON ERROR | %s | payload=%r | continuing", exc, body[:200])
        channel.basic_nack(delivery_tag=method.delivery_tag, requeue=False)
        return
    log.info("RECEIVED | queue=%s | temperature=%s C | humidity=%s %% | redelivered=%s",
             QUEUE_NAME, temperature, humidity, method.redelivered)
    channel.basic_ack(delivery_tag=method.delivery_tag)
```

```
channel.basic_qos(prefetch_count=1)
channel.basic_consume(queue=QUEUE_NAME,
                      on_message_callback=on_message,
                      auto_ack=False)
```

parse_telemetry giải mã UTF-8 và gọi json.loads. Hàm yêu cầu JSON object có hai trường temperature và humidity là số hữu hạn; giá trị độ ẩm phải nằm trong khoảng 0 đến 100. Consumer hiển thị hai trường cùng đơn vị đo, sau đó mới gọi basic_ack.

ACK cho biết consumer đã xử lý xong. Ready là số bản tin đang chờ được giao; Unacked là số bản tin đã giao nhưng chưa được xác nhận. Với consumer xử lý nhanh, hai chỉ số thường về 0 ngay sau mỗi lần nhận.

Nếu JSON sai hoặc thiếu trường hợp lệ, consumer in JSON ERROR và gọi basic_nack với requeue=False. Bản tin lỗi bị loại bỏ vì bài lab không cấu hình dead letter queue; consumer tiếp tục nhận bản tin tiếp theo. Cơ chế này tránh lặp vô hạn với một bản tin hỏng.

Khi consumer dừng, producer vẫn gửi vào queue. Các bản tin chưa giao chuyển thành Ready và được giữ trong queue. Khi consumer chạy lại, RabbitMQ chuyển các bản tin chờ cho consumer; sau khi xử lý và ACK, Ready và Unacked trở về 0.

Không nên khái quát rằng MQTT luôn mất bản tin khi subscriber vắng mặt: khả năng nhận lại còn phụ thuộc QoS, session và cấu hình broker. Thí nghiệm này trực tiếp chứng minh hành vi của queue RabbitMQ đang sử dụng.

4 Chạy hệ thống và kết quả kiểm tra

Yêu cầu môi trường là Docker Desktop chạy Linux containers. Không cần cài Python trên máy chủ. Chạy các lệnh sau trong PowerShell tại thư mục bài; chỉ copy .env ở lần đầu.

```
cd D:\IOT-asm\bai3\bai3_1
Copy-Item .env.example .env
docker compose up --build -d
docker compose ps
docker compose logs -f producer consumer
```

Mở http://localhost:15672, đăng nhập bằng tài khoản mẫu nhom16 / nhom16_lab_local, rồi chọn Queues and Streams và queue it4735.nhom16.telemetry. Tài khoản lấy từ .env; producer và consumer dùng hostname rabbitmq trong Docker. Ctrl+C ở màn hình xem log chỉ thoát theo dõi log.

Quy trình chụp kết quả: ghi nhận trạng thái gửi nhận; dừng consumer khoảng 30 giây; dừng producer để giữ nguyên số bản tin cho ảnh; chạy lại consumer để nhận hết rồi khởi động producer trở lại.

```
docker compose stop consumer
Start-Sleep -Seconds 30
docker compose stop producer
# Chup trang queue sau khi chi so cap nhat
docker compose start consumer
docker compose logs --since 2m consumer
# Chup queue da ve 0, sau do:
docker compose start producer
```

Kiểm tra thực tế bằng verify.ps1 trên Docker tại 10:55 ngày 23/10/2026. Kết quả gốc được lưu trong output/kiem_tra.json và output/kiem_tra.txt.

Trạng thái	Ready	Unacked	Consumer
Trước khi dừng	0	0	1
Dừng consumer khoảng 30 giây	6	0	0
Chạy lại và nhận hết	0	0	1

Kiểm tra bổ sung gửi chuỗi hello rồi JSON hợp lệ cho kết quả PASS: consumer báo lỗi bản tin đầu và tiếp tục hiển thị temperature=23 C, humidity=33 %. Bốn ảnh chụp thực tế được trình bày ở hai trang sau.

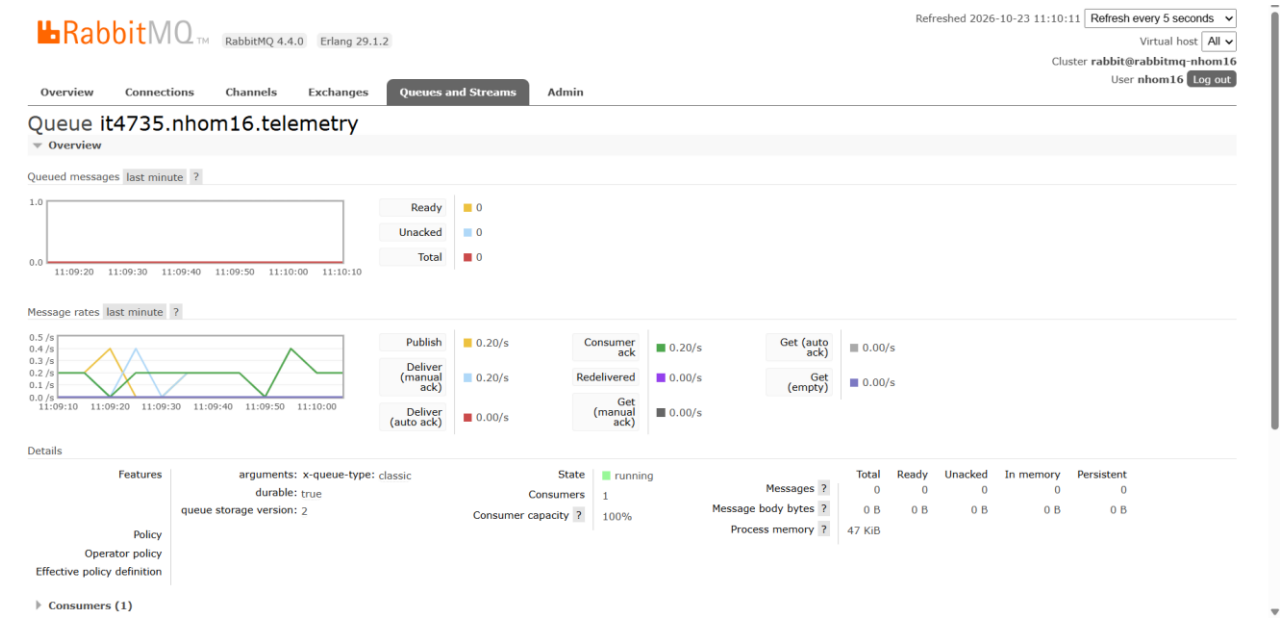
5 Ảnh chạy chương trình và trang queue

Các ảnh dưới đây ghi lại log chương trình và trạng thái queue trong quá trình thử nghiệm.

```
D:\IOT-asm\bai3\bai3_1>docker compose ps
NAME                    IMAGE                    COMMAND                  SERVICE    CREATED          STATUS                    PORTS
bai3-nhom16-consumer-1   bai3-nhom16-app:local    "python -u consumer.…"   consumer   15 minutes ago   Up 12 minutes
bai3-nhom16-producer-1   bai3-nhom16-app:local    "python -u producer.…"   producer   15 minutes ago   Up 11 minutes
bai3-nhom16-rabbitmq-1   rabbitmq:4.4.0-management   "docker-entrypoint.s…"   rabbitmq   15 minutes ago   Up 15 minutes (healthy)   127.0.0.1:5672->5672/tcp, 127.0.0.1:15672->15672/tcp

D:\IOT-asm\bai3\bai3_1>docker compose logs --tail 12 producer consumer
consumer-1  | 2026-10-23 04:07:29,739 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=25.8 C | humidity=74.6 % | redelivered=False
consumer-1  | 2026-10-23 04:07:34,768 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=27.0 C | humidity=67.5 % | redelivered=False
consumer-1  | 2026-10-23 04:07:39,800 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=23.4 C | humidity=57.3 % | redelivered=False
consumer-1  | 2026-10-23 04:07:44,819 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=25.9 C | humidity=52.7 % | redelivered=False
consumer-1  | 2026-10-23 04:07:49,830 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=32.4 C | humidity=64.7 % | redelivered=False
consumer-1  | 2026-10-23 04:07:54,843 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=26.5 C | humidity=72.7 % | redelivered=False
consumer-1  | 2026-10-23 04:07:59,860 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=27.7 C | humidity=77.9 % | redelivered=False
consumer-1  | 2026-10-23 04:08:04,875 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=26.7 C | humidity=68.3 % | redelivered=False
consumer-1  | 2026-10-23 04:08:09,901 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=23.5 C | humidity=73.3 % | redelivered=False
consumer-1  | 2026-10-23 04:08:14,932 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=29.8 C | humidity=44.6 % | redelivered=False
consumer-1  | 2026-10-23 04:08:19,949 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=23.8 C | humidity=52.7 % | redelivered=False
consumer-1  | 2026-10-23 04:08:24,972 | RECEIVED | queue=it4735.nhom16.telemetry | temperature=29.6 C | humidity=76.2 % | redelivered=False
producer-1  | 2026-10-23 04:07:29,740 | SENT #137 | queue=it4735.nhom16.telemetry | {"temperature": 25.8, "humidity": 74.6}
producer-1  | 2026-10-23 04:07:34,769 | SENT #138 | queue=it4735.nhom16.telemetry | {"temperature": 27.0, "humidity": 67.5}
producer-1  | 2026-10-23 04:07:39,803 | SENT #139 | queue=it4735.nhom16.telemetry | {"temperature": 23.4, "humidity": 57.3}
producer-1  | 2026-10-23 04:07:44,821 | SENT #140 | queue=it4735.nhom16.telemetry | {"temperature": 25.9, "humidity": 52.7}
producer-1  | 2026-10-23 04:07:49,830 | SENT #141 | queue=it4735.nhom16.telemetry | {"temperature": 32.4, "humidity": 64.7}
producer-1  | 2026-10-23 04:07:54,844 | SENT #142 | queue=it4735.nhom16.telemetry | {"temperature": 26.5, "humidity": 72.7}
producer-1  | 2026-10-23 04:07:59,861 | SENT #143 | queue=it4735.nhom16.telemetry | {"temperature": 27.7, "humidity": 77.9}
producer-1  | 2026-10-23 04:08:04,876 | SENT #144 | queue=it4735.nhom16.telemetry | {"temperature": 26.7, "humidity": 68.3}
producer-1  | 2026-10-23 04:08:09,902 | SENT #145 | queue=it4735.nhom16.telemetry | {"temperature": 23.5, "humidity": 73.3}
producer-1  | 2026-10-23 04:08:14,934 | SENT #146 | queue=it4735.nhom16.telemetry | {"temperature": 29.8, "humidity": 44.6}
producer-1  | 2026-10-23 04:08:19,950 | SENT #147 | queue=it4735.nhom16.telemetry | {"temperature": 23.8, "humidity": 52.7}
producer-1  | 2026-10-23 04:08:24,973 | SENT #148 | queue=it4735.nhom16.telemetry | {"temperature": 29.6, "humidity": 76.2}
```

Hình 1 RabbitMQ, producer và consumer đang chạy; log hiển thị nhiệt độ và độ ẩm.

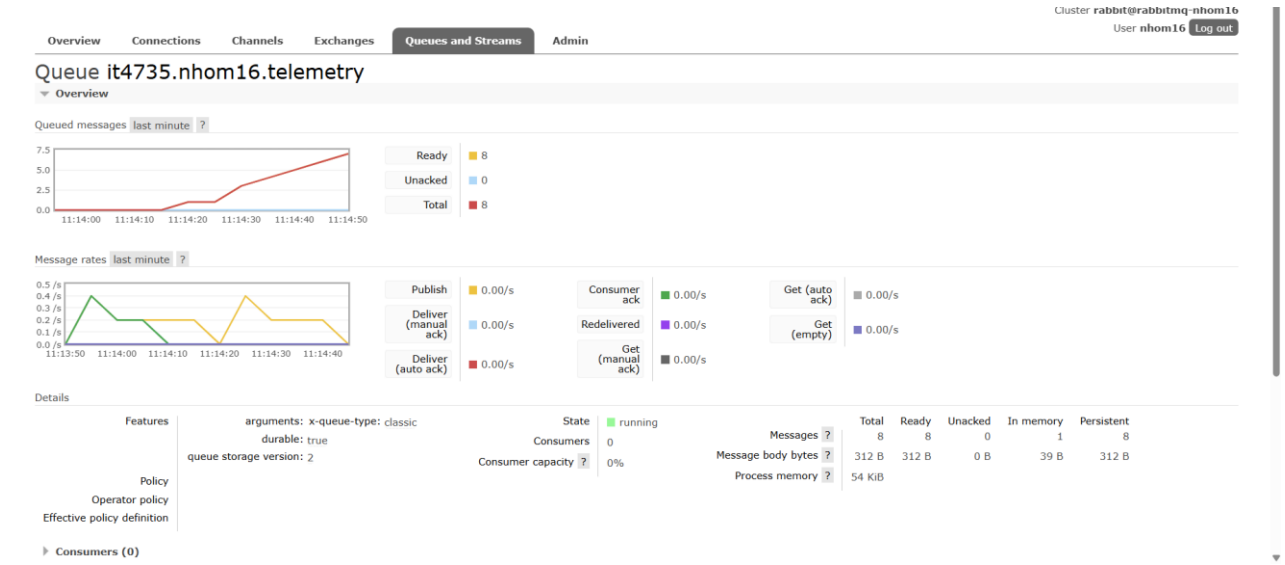


Hình 2 Trang quản trị queue nhóm 16 khi hệ thống gửi và nhận dữ liệu bình thường.

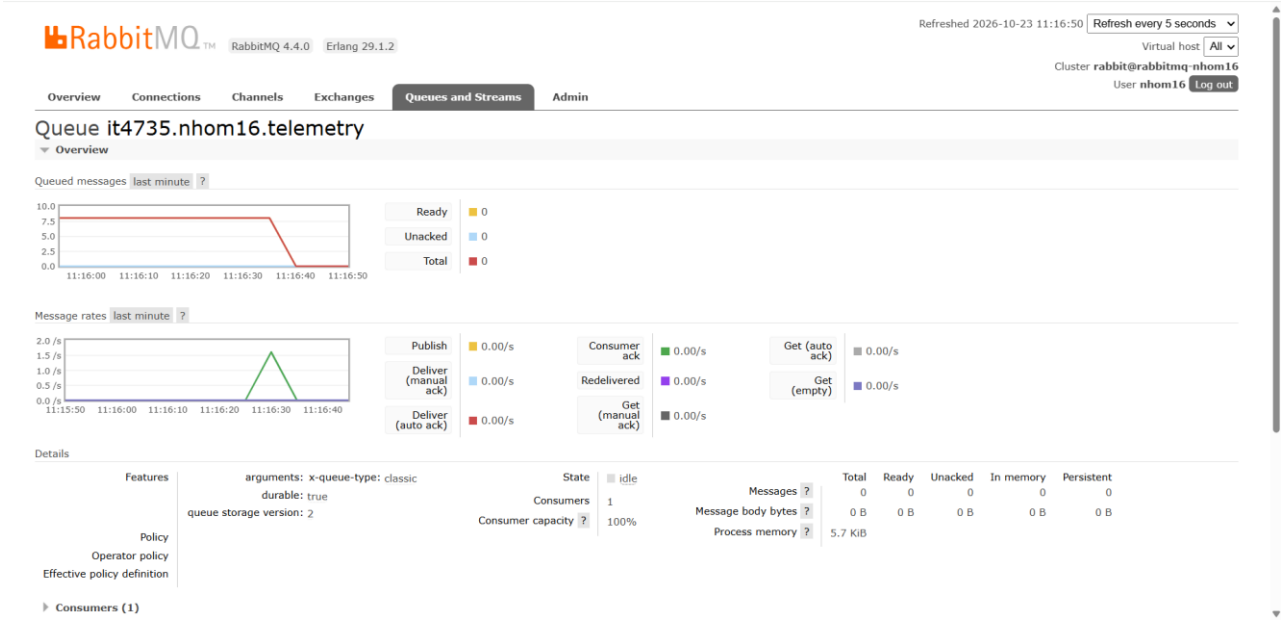
Ảnh chụp giữ được tên queue và các số Ready, Unacked, Consumers. Với chu kỳ 5 giây, biểu đồ tốc độ có thể tạm bằng 0 giữa hai bản tin.

6 Ảnh kiểm chứng lưu bản tin trong queue

Dừng consumer khoảng 30 giây trong khi producer tiếp tục gửi. Sau đó dừng producer để có thời gian chụp ảnh, rồi chạy lại consumer và quan sát queue được xử lý hết.



Hình 3 Queue tích bản tin khi consumer dừng; ảnh chụp thực tế ghi nhận Ready = 8, Consumers = 0.



Hình 4 Consumer nhận các bản tin tồn đọng; Ready = 0, Unacked = 0, Consumers = 1.

Nhận xét: thí nghiệm cho thấy producer và consumer hoạt động độc lập về thời gian nhờ queue. Việc dừng consumer không làm mất các bản tin đã được broker tiếp nhận và đang chờ trong lần thử này.